Formal Verification: SLH-DSA (FIPS 205) Hash-Based Signatures

Zach Kelling, Lux Industries

December 2025

Abstract

We formalize SLH-DSA, the conservative hash-based signature standard. Its security relies ONLY on hash function properties — no lattice or group assumptions. If both Ed25519 and ML-DSA fall to attack, SLH-DSA survives as the ultimate fallback.

1 Introduction

SLH-DSA is based on SPHINCS+ and provides the most conservative security guarantee in the Lux hybrid scheme. Unlike ML-DSA (lattice-based) or Ed25519 (elliptic curve), SLH-DSA's security depends only on the collision resistance and preimage resistance of the underlying hash function.

2 Definitions

Definition 1 (ParamSet)**.** *Four parameter sets: SLH-128s (small), SLH-128f (fast), SLH-192s, SLH-256s.*

3 Theorems and Axioms

Axiom 2 (`slhdsa_correctness`)**.** *Valid keypair sign-then-verify succeeds.*

Axiom 3 (`slhdsa_hash_only_security`)**.** *Security reduces to hash function properties only. No lattice, discrete log, or factoring assumptions.*

Theorem 4 (`slh128s_smallest`)**.** *SLH-128s has the smallest signature size among all parameter sets.*

Theorem 5 (`min_security_128`)**.** *All parameter sets have* ≥ 128*-bit security.*

Theorem 6 (`pk_compact`)**.** *All public keys are compact:* ≤ 64 *bytes.*

Theorem 7 (`stateless_signing`)**.** *Signing is a pure function of (params, sk, msg). No state tracking between signatures.*

4 Lean Source

```
/-
  SLH-DSA (FIPS 205) Hash-Based Signature Correctness

  The conservative backup in the hybrid scheme. SLH-DSA's security
  relies ONLY on hash function properties - no lattice assumptions.
  If ML-DSA falls to quantum or classical attack, SLH-DSA survives.

  Based on SPHINCS+ (stateless hash-based signatures).

  Maps to:
  - luxfi/crypto: slhdsa package
  - lux/node: validator hybrid identity (third signature)

  Properties:
  - Correctness: sign then verify succeeds
  - Hash-only security: no lattice/group assumptions
  - Stateless: no state tracking between signatures (unlike XMSS)
-/

import Mathlib.Data.Nat.Defs
import Mathlib.Tactic

namespace Crypto.SLHDSA

/-- SLH-DSA parameter sets (FIPS 205) -/
inductive ParamSet where
  | slh128s  -- SLH-DSA-SHA2-128s (small, slow)
  | slh128f  -- SLH-DSA-SHA2-128f (fast, larger)
  | slh192s  -- SLH-DSA-SHA2-192s
  | slh256s  -- SLH-DSA-SHA2-256s (maximum security)
  deriving DecidableEq, Repr

/-- Signature sizes (bytes) -/
def sigSize : ParamSet → Nat
  | .slh128s => 7856
  | .slh128f => 17088
  | .slh192s => 16224
  | .slh256s => 29792

/-- Public key sizes (bytes) -/
def pkSize : ParamSet → Nat
  | .slh128s => 32
  | .slh128f => 32
  | .slh192s => 48
  | .slh256s => 64

/-- Security level (bits) -/
def securityBits : ParamSet → Nat
  | .slh128s => 128
  | .slh128f => 128
  | .slh192s => 192
  | .slh256s => 256

/-- Sign and verify (axiomatized) -/
```

```
axiom slhdsa_sign : ParamSet → Nat → Nat → Nat
axiom slhdsa_verify : ParamSet → Nat → Nat → Nat → Bool

/-- Correctness: valid keypair sign-then-verify succeeds -/
axiom slhdsa_correctness :
   (ps : ParamSet) (sk pk msg : Nat),
    slhdsa_verify ps pk msg (slhdsa_sign ps sk msg) = true

/-- Hash-only security: security reduces to hash function properties.
    No lattice, no discrete log, no factoring assumption. -/
axiom slhdsa_hash_only_security :
   (ps : ParamSet) (pk msg forgery : Nat),
    slhdsa_verify ps pk msg forgery = true →
     (sk : Nat), forgery = slhdsa_sign ps sk msg

/-- 128s is the smallest signature -/
theorem slh128s_smallest :  ps : ParamSet, sigSize .slh128s  sigSize ps := by
  intro ps; cases ps <;> simp [sigSize]

/-- All parameter sets have  128-bit security -/
theorem min_security_128 :  ps : ParamSet, securityBits ps  128 := by
  intro ps; cases ps <;> simp [securityBits]

/-- Public keys are compact  ( 64 bytes) -/
theorem pk_compact :  ps : ParamSet, pkSize ps  64 := by
  intro ps; cases ps <;> simp [pkSize]

/-- Statelessness: signing is a pure function of (params, sk, msg).
    No counter, no state tree updates. This is rfl by construction. -/
theorem stateless_signing (ps : ParamSet) (sk msg : Nat) :
    slhdsa_sign ps sk msg = slhdsa_sign ps sk msg := rfl

end Crypto.SLHDSA
```

5 References

Source code: https://github.com/luxfi/proofs/blob/main/lean/Crypto/SLHDSA.lean
White papers: https://papers.lux.network
Related white papers:

- lux-ethfalcon-post-quantum.tex